JÖNKÖPING UNIVERSITY
School of Engineering

LABORATION 3 – IDS/IPS

Tredje laborationen i kursen *Nätverkssäkerhet* tar upp *Intrusion Detection Systems (IDS)* samt *Intrusion Prevention Systems (IPS)*. I denna labb ska vi sätta upp den vanligaste IDS/IPS-mjukvaran i världen som är Snort. Avslutningsvis undersöker vi Cisco Firepower.

En brandvägg följer ett antal fördefinierade regler för hur trafik ska få flöda (access-listor) och agera (protokollinspektion). Ibland kan det dock finnas sårbarheter senare i kedjan, exempelvis kan vi ha en webbserver som man ska få nå publikt på port 80, men där mjukvaran inte är uppdaterad. I detta fall kan en IPS som känner till sårbarheten stoppa attacker mot servern trots att trafiken mot port 80 är godkänd av brandväggen. En IDS/IPS (ibland skrivet IDPS) tar vid där brandväggens funktionalitet slutar. I moderna brandväggar finns ofta IPS-funktionen inbyggd, exempelvis FortiGate och Firepower.

1 Uppgift

Uppgiften förutsätter deltagande på veckans föreläsning samt genomläsning av kapitel 11 och 12 innan laborationen påbörjas.

I labbrapporten ska gruppen (min 2 personer, max 3 personer) bevisa att de har utfört alla fetmarkerade uppgifter i gula rutor, detta kan exempelvis vara med hjälp av skärmbilder, förklaringar, konfiguration och svar på frågor. Om inget är specificerat väljer ni själva hur ni vill bevisa att ni har utfört uppgiften. Om laborationen utförs utanför ordinarie laborationspass är det tillåtet att arbeta och lämna in enskilt. Säkerställ att skärmbilder är beskurna på ett bra sätt, inte för mycket och inte för lite. Såväl skärmbilder som täcker hela skärmen som skärmbilder där man endast ser en enstaka rad duger inte.

Laborationen innehåller följande delar:

1. Snort
2. Firepower
3. *Frivillig: Host-IPS*

1.1 Snort

Snort är en mjukvara baserad på öppen källkod som utvecklades av Martin Roesch som senare grundade företaget Sourcefire. Företaget blev 2013 uppköpt av Cisco. Det är den mest använda IDPS-mjukvaran är världen. Snort kan köras stand-alone på en server, på vissa routrar eller som del i en hårdvaruappliance. Läs gärna mer om Snort här: https://www.snort.org/

Det vanligaste sättet att köra Snort är via ett Linux-baserat operativsystem. Rekommendationen för labben är senaste versionen av Ubuntu Server. Du får givetvis använda en annan distribution om du önskar, dock kan instruktioner skilja. Om en *netinstall* används måste NAT-läget användas i hypervisorn. Rekommendationen är att ladda ned via en svensk mirror.

1. Installera Ubuntu Server i VirtualBox och aktivera SSH-åtkomst (t.ex. OpenSSH-server) till servern.

Ställ in nätverksinställningar på sådant sätt att servern är nåbar från Kali Linux, exempelvis genom ett *NAT Network* som delas med Kali Linux eller bryggat läge.

För att skapa ett nytt NAT Network, gå in i *File -> Tools -> Network Manager* i VirtualBox. Ställ in så att både Kali och Ubuntu sitter i samma nätverk, får DHCP och kan nå Internet.

Logga in på servern via SSH, och installera Snort enligt nedanstående instruktioner. Börja med att uppdatera paketlistor. Installera sedan Snort via apt. Om ni hellre vill installera Snort via källkod hänvisas ni till **Appendix A**.

```
apt update
apt install snort
```

Under installationen ska ett par saker ställas in, t.ex. interface och nätverk. Vilket interface som ska användas ser du med verktyget *ip addr*. I kommande exempel används **enp0s3**.

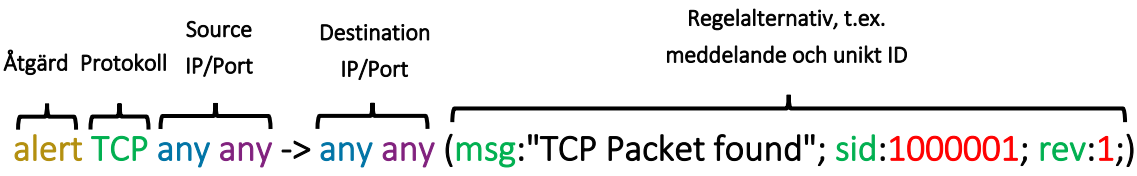
2. Verifiera att Snort är installerat med kommandot snort -V. **Ta en skärmbild**

Nu är det dags att skapa en egen regel, och detta görs under **/etc/snort/rules/local.rules**

```
vim /etc/snort/rules/local.rules
```

I editeringsverktyget *vim* kommer man till editeringsläge genom att klicka INSERT. Stäng ned vim genom att trycka på ESC och skriv **:wq** (sparar och stänger vim). Det är givetvis okej att använda en annan textredigerare, t.ex. *nano,* för detta.

En enkel regel är uppbyggd enligt följande parametrar:



Det finns också flera parametrar som är valfria, en vanlig är den för ”content” som kan användas för att identifiera ett visst innehåll i ett paket.

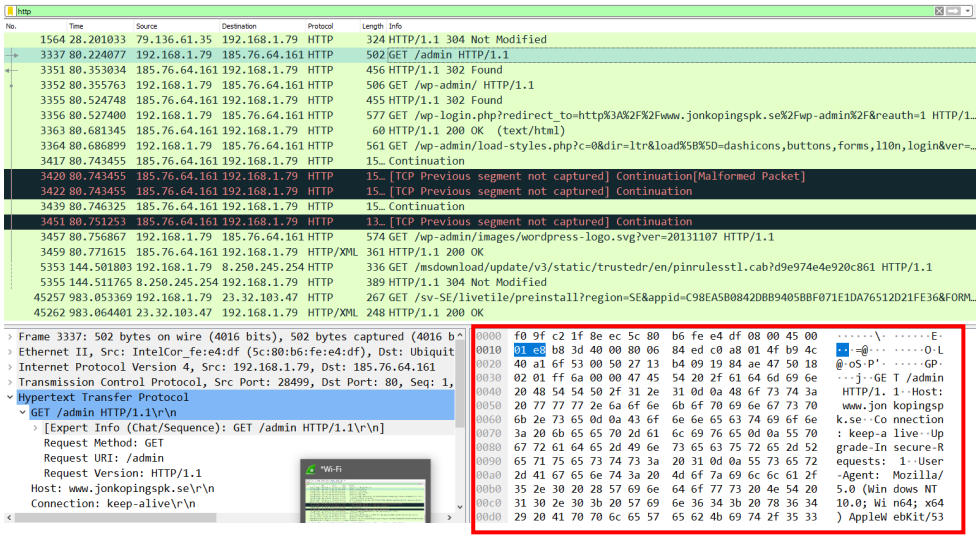
3. Skapa en simpel Snort-regel som matchar ett ping-paket. Starta igång Snort i konsol-läge med *snort -A console -i enp0s3 -c /etc/snort/snort.conf* **(Notera:** Byt ut *enp0s3* mot det interface som ska testas mot**). Testa regeln genom att skicka en ping från Kali mot servern. Ta en skärmbild både på regeln samt loggmeddelandet. Förutom din egen regel kommer även standardreglerna i Snort matcha trafiken. Eventuellt kommer även en del ICMPv6-trafik att loggas om IPv6 inte är avaktiverat.**

Installera Apache på Snort-servern med **apt install apache2**. Verifiera att ni når webbservern från en annan dator (t.ex. Kali).

4. Skapa en lite mer ”avancerad” regel i Snort som matchar när någon försöker komma åt webbserverns underkatalog /admin **som är tänkt att bara användas i det interna nätverket. Ta en skärmbild både på regeln samt genererat loggmeddelandet.**

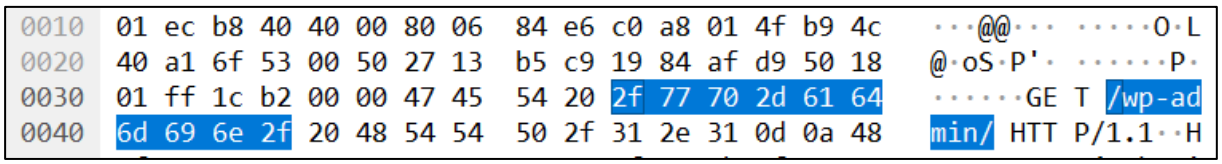
Hint: Använd regelalternativet *”content”* för att särskilja trafiken.

Väldigt många attacker går att filtrera bara genom att använda *content*-parametern på rätt sätt. Ofta vill man dock inte bara matcha på ren text, utan på en del av ett paket. Ett sätt att få fram denna information om ett skadligt paket är att fånga trafiken med hjälp av Wireshark. Ni har säkert noterat i programvaran att (som standard) presenteras paketets rådata i den nedre högre halvan av skärmen (se *Figur 1* nedan).



Figur 1. Standardskärm för Wireshark.

Ett sätt att skapa regler i Snort är att markera rådatan och kopiera hexkoden (*hex code* eller *hex bytes*) för den data som man är intresserad av. I *Figur 2* nedan ser vi ett exempel på hur vi får ut hexkoden för */wp-admin* som blir *2f 77 70 2d 61 64 6d 69 6e 2f.* Anledningen till att två hexadecimala tal grupperas är för att de tillsammans blir en byte (ett hexadecimalt tal är 4 bitar).



Figur 2. Rådata för ett paket i Wireshark.

För att skapa regeln skriver man den enligt följande: **content:”|2f 77 70 2d 61 64 6d 69 6e 2f|”**. Denna regel kommer alltså matcha textsträngen */wp-admin*.

5. Skapa en till regel i Snort som matchar när någon försöker komma åt webbserverns underkatalog /hextest**. Detta måste lösas med hjälp av hexkod. Ta en skärmbild både på regeln samt genererat loggmeddelandet.**

Hint: Använd en konverterare för ASCII-to-Hex eller generera koden via Wireshark.

Dags att testa lite andra Snort-regler. Vi startar igång så att Snort rapporterar i realtid till terminalen för att vi ska kunna se direkt vilka intrång den detekterar (vanligast i en produktionsmiljö är dock att Snort rullar kontinuerligt i bakgrunden för att samla loggar om, eller stoppa, intrångsförsök).

```
snort -A console -i enp0s3 -c /etc/snort/snort.conf
```

Notera: Byt ut *enp0s3* mot det interface som ska testas mot

OBS: NU ÄR DET DAGS ATT UTFÖRA PENETRATIONSTESTNING. UTFÖR ENDAST DENNA TYP AV TESTER PÅ ENHETER OCH NÄTVERK SOM NI SJÄLV STYR ÖVER ELLER SOM NI HAR FÅTT TILLÅTELSE ATT TESTA. TESTA ALDRIG UT MOT INTERNET DÅ DETTA KAN VARA OLAGLIGT.

I Kali Linux ska vi först testa att utföra lite olika typer av portskanningar via *nmap*.

```
nmap -sF [IP-adress till Snort]
nmap -sX [IP-adress till Snort]
```

I Kali Linux ska vi använda *Metasploit Framework (MSF)* för att testa några av standardreglerna i Snort. Metasploit körs effektivast i terminalvarianten *msfconsole*, sök upp Metasploit via Kalis startmeny för att *msfconsole* ska starta med databasstöd. Börja med att utföra en vanlig portskanning från Kali Linux till Snort med följande:

```
msf6 > db_nmap -A -T4 [IP-adress till Snort]
```

6. Ta skärmbilder på de olika portskanningarna (från Kali), samt de paket som Snort eventuellt identifierar som skadliga (från Snort).

Notera: Loggmeddelanden om UPnP samt IPv6 har sannolikt inte med detta att göra, rekommendationen är att avaktivera IPv6 under labben.

Nu är det dags att testa lite enkla attacker. En attack som oskyddade servrar kan vara känsliga mot är att ändra i SNMP SET-parametrar. I *msfconsole* skriver ni:

```
msf6 > use auxiliary/scanner/snmp/snmp_set
msf6 > show options
Module options (auxiliary/scanner/snmp/snmp_set):
  Name          Current Setting  Required  Description
  ----          ---------------  --------  -----------
  COMMUNITY     public           yes       SNMP Community String
  OID                            yes       The object identifier (numeric notation)
  OIDVALUE                       yes       The value to set
  RETRIES       1                yes       SNMP Retries
  RHOSTS                         yes       The target address range or CIDR identifier
```

Oftast måste man ställa in **RHOSTS** (och eventuellt något mer), detta beror dock på vilken typ av attack det är som ska utföras. I detta fall måste vi även ställa in **OID** och **OIDVALUE** som är specifikt för just SNMP. Sedan sätter man i gång attacken med *run* eller *exploit*.

msf6 auxiliary(scanner/snmp/snmp_set) > *set RHOSTS 193.10.160.✗*

msf6 auxiliary(scanner/snmp/snmp_set) > *set OID 1.3.6.1.4.1.2021.11.9* (sätt valfri OID)

msf6 auxiliary(scanner/snmp/snmp_set) > *set OIDVALUE 1* (sätt valfritt värde)

msf6 auxiliary(scanner/snmp/snmp_set) > *exploit*

7. Ta en skärmbild på de SNMP-paket som Snort identifierar som potentiellt skadliga.

Som ni märker är det ett visst merarbete att sitta i CLI och arbeta med Snort. Både för att kolla av loggar, och för att ställa in lämpliga regler och åtgärder. Många delar går att styra via ett grafiskt gränssnitt och det vanligaste är BASE. Om ni vill testa det grafiska gränssnittet hittar ni instruktioner här: https://www.securityarchitecture.com/learning/intrusion-detection-systems-learning-with-snort/installing-base/. Detta ingår inte i laborationen.

En utav de viktigaste delarna när man kör en IPS är att hålla den uppdaterad för nya sårbarheter, ofta kommer attacksignaturer väldigt fort till kända sårbarheter och det är oftast i början som de utnyttjas. Ett tillägg för Snort som håller attacksignaturerna uppdaterade är PulledPork som ni kan läsa mer om här: https://allcloud.io/blog/configure-snort-automatic-rules-updating-via-pulledpork/. Detta ingår inte i laborationen.

1.2 Firepower

Företaget SourceFire som utvecklade Snort blev som tidigare nämnt uppköpt av Cisco under 2013. Sedan dess har Cisco ersatt IOSIPS-funktionen och IPS i Cisco ASA med Firepower. IPS-motorn i Firepower bygger på Snort. Andra leverantörer har sina egna specifika lösningar för IPS.

8. Undersök och skriv en kort redogörelse (5-6 meningar) över IPS-funktionaliteten i Firepower

1.3 Frivillig: Host-IPS

Tekniskt sett är det inga problem att köra Snort på sin egen dator eller på en server, men problemet är att mjukvaran endast skyddar mot nätverksattacker. Det finns därför IPS-lösningar som innehåller attacksignaturer som passar bättre till host- och server-enheter. En utav de vanligaste programvarorna för detta är OSSEC som är baserad på öppen källkod. Testa gärna OSSEC på din labbdator eller Ubuntu-servern. Läs mer här: https://www.ossec.net/

Appendix A – Installera Snort från källkod

```
apt-get install -y build-essential libpcap-dev libpcre3-dev          # Installera dependencies
apt-get install -y libdumbnet-dev bison flex zlib1g-dev              och uppdatera
apt-get install -y liblzma-dev openssl libssl-dev libnghttp2-dev luajit
apt-get update

mkdir ~/snort_src                                                    # Skapa och gå till katalog
cd ~/snort_src

wget https://www.snort.org/downloads/snort/daq-2.0.6.tar.gz          # Ladda ned Snort-filerna
wget https://www.snort.org/downloads/snort/snort-2.9.13.tar.gz

tar xvzf daq-2.0.6.tar.gz                                            # Packa upp och installera
cd daq-2.0.6                                                         DAQ (Data Acquisition library)
./configure && make && sudo make install                             som är ett krav för Snort

cd ~/snort_src                                                       # Packa upp, kompilera och
tar xvzf snort-2.9.13.tar.gz                                         installera Snort
cd snort-2.9.13
./configure --enable-sourcefire --disable-open-appid && make && sudo make install
```

Om något felmeddelande dök upp under installationen hittar du antagligen en lösning i manualen Snort 2.9.9.x on Ubuntu 14 and 16 eller på sidan http://sublimerobots.com/. Det går också att installera Snort via pakethanteraren (apt-get), men detta är ett utmärkt tillfälle att öva på lite grundläggande Linux-administration. Om man kompilerar källkoden själv blir programmet dessutom något effektivare att köra, något som är särskilt viktigt när Snort används som IPS i en produktionsmiljö. Innan vi startar igång Snort ska vi uppdatera delade bibliotek.

```
ldconfig
```

För att kunna köra Snort från var som helst i terminalen ska vi också skapa en *symlink* (liknar genvägar) till *sbin* som hanterar detta (gäller inte om du har installerat Snort via apt).

```
ln -s /usr/local/bin/snort /usr/sbin/snort
snort -V (för att testa installationen samt symlänken)
```

Snort bygger på att följa fördefinierade regler, när man själv kompilerar Snort skapas inte dessa regler automatiskt därför måste vi skapa ett antal filer och kataloger och flytta de rekommenderade filerna från vår uppackade Snort-fil.

```
mkdir /etc/snort                                          # Skapa kataloger
mkdir /etc/snort/rules
mkdir /etc/snort/rules/iplists
mkdir /etc/snort/preproc_rules
mkdir /usr/local/lib/snort_dynamicrules
mkdir /etc/snort/so_rules

touch /etc/snort/rules/iplists/black_list.rules           # Skapa filer för regler
touch /etc/snort/rules/iplists/white_list.rules
touch /etc/snort/rules/local.rules
touch /etc/snort/sid-msg.map

mkdir /var/log/snort                                      # Skapa filer för loggar
mkdir /var/log/snort/archived_logs

cd ~/snort_src/snort-2.9.13/etc/                          # Kopiera konfigurationsfiler
cp *.conf* /etc/snort
cp *.map /etc/snort
cp *.dtd /etc/snort
cd ~/snort_src/snort-2.9.13/src/dynamic-
preprocessors/build/usr/local/lib/snort_dynamicpreprocessor/
cp * /usr/local/lib/snort_dynamicpreprocessor/
```

Eftersom vi är inloggade som *root* kommer Snort att köras med full behörighet över hela systemet. Detta är inte optimalt i en produktionsmiljö med höga säkerhetskrav. Du rekommenderas att läsa igenom kapitel 11 i Snort 2.9.9.x on Ubuntu 14 and 16 som går igenom hur man skapar användare med tillhörande grupp i Ubuntu, samt hur man ändrar behörigheterna för hur Snort ska köras på säkraste sättet. Detta är inte obligatoriskt för denna laboration.

De reglerna som Snort använder regleras i **/etc/snort/snort.conf**. Ett par ändringar ska göras i konfigurationsfilen.

```
vim /etc/snort/snort.conf
```

Här är det vanligt att ändra variablerna **HOME_NET** och **EXTERNAL_NET**, i detta fallet kan vi dock behålla dem på *any*. Ställ in variabeln **RULE_PATH** (rad 104) till värdet *rules*, ställ även in nedanstående variabler enligt bilden (klicka INSERT för att komma till editeringsläge).

```
var RULE_PATH rules
var SO_RULE_PATH so_rules
var PREPROC_RULE_PATH preproc_rules
var WHITE_LIST_PATH rules/iplists
var BLACK_LIST_PATH rules/iplists
```

Stäng ned *vim* genom att trycka på ESC och skriva **:wq** (sparar och stänger vim). För att verifiera att konfigurationsfilen är godkänd körs ett testkommando.

```
snort -T -i enp0s3 -c /etc/snort/snort.conf
```

Notera: Byt ut enp0s3 mot det interface du vill testa mot (kolla ifconfig)

Antagligen kommer verifikationen inte att gå igenom, vanligast är att det saknas flera regelfiler som finns med i konfigurationen. Ett enkelt sätt att kommentera ut dessa rader finns nedan.

```
sed -i "s/include \$RULE\_PATH/#include \$RULE\_PATH/" /etc/snort/snort.conf
```

För att ställa in att Snort använder sig av reglerna i filen **local.rules** måste vi gå in i konfigurationsfilen igen (**/etc/snort/snort.conf**) och avkommentera rad 541.

```
include $RULE_PATH/local.rules
```